Natural Language Processing (NLP)

Unit 3

Syntax II

UNIT – III

Syntax II: Models for Ambiguity Resolution in Parsing, Multilingual Issues

Semantic Parsing I: Introduction, Semantic Interpretation, System Paradigms, Word Sense.

MODELS FOR AMBIGUITY RESOLUTION IN PARSING

Explain the Models for Ambiguity Resolution in Parsing.

Ambiguity is a pervasive problem in syntactic parsing, where a single sentence can have multiple valid structural analyses. Models for ambiguity resolution are crucial for choosing the most plausible parse from an exponentially large number of alternatives. These models achieve this by assigning a score or probability to each possible parse tree. The primary models discussed are Probabilistic Context-Free Grammars, Generative Models, and Discriminative Models.

1. Probabilistic Context-Free Grammars (PCFG)

A Probabilistic Context-Free Grammar (PCFG) is an extension of a standard CFG where every grammar rule is assigned a probability. This allows the model to calculate a probability for each parse tree, with the highest-scoring tree being the most plausible.

The probability of a derivation is the product of the probabilities of all CFG rules used. The probability of a rule $N \rightarrow \alpha$ is conditioned on its left-hand side non-terminal N. For any non-terminal, the sum of probabilities of all its possible expansions must be 1:

$$1=\sum\alpha P(N\rightarrow\alpha)1=\alpha\sum P(N\rightarrow\alpha)$$

Example: Resolving PP-Attachment Ambiguity

Consider the ambiguous sentence *"John bought a shirt with pockets"*. A PCFG can resolve whether "with pockets" modifies "bought" (a verb phrase) or "shirt" (a noun phrase) by comparing the probabilities of the attachment rules:

- VP → V NP | VP PP
- NP → 'John' | 'pockets' | D N | NP PP

If the PCFG assigns P(NP → NP PP) = 0.5 and P(VP → VP PP) = 0.1, the parse where "with pockets" attaches to "shirt" (NP) will be preferred due to its higher probability.

Deriving Probabilities from a Treebank:
PCFG probabilities are learned by counting rule frequencies in a hand-annotated corpus (a treebank).
Diagram: Example Treebank Trees (t1, t2, t3)

If t1 appears 10 times, t2 20 times, and t3 50 times, the rule probabilities are calculated as follows:

- P(S → AB) = 10 / (10+20+50) = 0.125
- P(S → AC) = 20 / (10+20+50) = 0.25
- P(S → C) = 50 / (10+20+50) = 0.625

Using the PCFG for Parsing:
For a new input like "a a a a", the model can find the most likely parse even if that parse never appeared in the treebank.

Diagram: Two Parses for "a a a a"

- p1 = P(S → AB) × P(A → aa) × P(B → aa) = 0.125 × 0.334 × 0.285 = 0.01189
- p2 = P(S → AC) × P(A → a) × P(C → aaa) = 0.25 × 0.667 × 0.714 = 0.119

Since p2 > p1, the second parse is chosen as the most likely, demonstrating how a PCFG resolves ambiguity. The context-free nature of PCFGs, however, leads to independence assumptions that can be problematic.

---

2. Generative Models for Parsing

Generative models provide a more general framework for probabilistic parsing. Here, a parse tree y for a sentence x is defined by a sequence of derivation decisions $D = d_1, ..., d_n$. The joint probability of the sentence and its parse is the product of the probabilities of each decision, conditioned on the previous decisions (the **history**).

$$P(x,y)=P(d1,...,dn)=\prod i=1nP(di|d1,...,di-1)P(x,y)=P(d1,...,dn)=i=1\prod nP(di|d1,...,di-1)$$

Because the history can be infinitely large, a simplifying assumption is made by mapping the history to a finite set of equivalence classes using a function Φ.

$$P(d1,...,dn)=\prod i=1nP(di|\Phi(d1,...,di-1))P(d1,...,dn)=i=1\prod nP(di|\Phi(d1,...,di-1))$$

This Φ function maps the history H_i to a set of k feature functions:

$$P(d1,...,dn)=\prod i=1nP(di|\phi 1(Hi),...,\phi k(Hi))P(d1,...,dn)=i=1\prod nP(di|\phi 1(Hi),...,\phi k(Hi))$$

PCFGs are a simple type of generative model, but their strict independence assumptions often lead to less accurate, though plausible-scoring, parses.

---

3. Discriminative Models for Parsing

Unlike generative models that model the joint probability $P(x,y)$, discriminative models model the conditional probability $P(y|x)$ directly. This allows them to use a rich, arbitrary set of features from the parse tree without being constrained by the independence assumptions of a CFG.

Global Linear Model Framework:

- An input x and an output parse y are mapped to a d-dimensional **feature vector** $\Phi(x,y)$.
- A **weight vector** w assigns a weight to each feature.
- The score of a parse is the dot product: $\text{score}(x,y) = \Phi(x,y) \cdot w$.
- The function $GEN(x)$ generates all possible parses for x.
- The best parse y^* is the one with the highest score:

$$F(x)=\text{argmax}\,y\in GEN(x)\Phi(x,y)\cdot wF(x)=y\in GEN(x)\text{argmax}\Phi(x,y)\cdot w$$

The Perceptron Algorithm for Learning:
The weight vector w is learned from training data using an algorithm like the **perceptron**. This is an online learning algorithm that processes one training example (x_i, y_i) at a time.

Algorithm: The Original Perceptron Learning Algorithm

Inputs: Training Data $((x_1, y_1), ..., (x_m, y_m))$; number of iterations T
Initialization: Set w = 0
Algorithm:
1: for t = 1, ..., T do
2: for i = 1, ..., m do

3: Calculate y_i', where $y_i' = \text{argmax}_{y \in GEN(x_i)} \Phi(x_i, y) \cdot w$
4: if $y_i' \neq y_i$ then
5: $w = w + \Phi(x_i, y_i) - \Phi(x_i, y_i')$
6: end if
7: end for
8: end for
Output: The updated weight parameter vector w

Explanation: The algorithm initializes weights to zero. For each training example, it finds the current best-scoring parse y_i'. If this parse is incorrect (i.e., $y_i' \neq y_i$), it updates the weight vector by adding the feature vector of the correct parse (y_i) and subtracting the feature vector of the incorrect parse (y_i'). This update increases the score of the correct parse and decreases the score of the incorrect one for that example.

Multilingual Issues

Explain various Multilingual Issues

When developing syntactic parsers, it is crucial to recognize that the fundamental assumptions valid for one language, like English, may not apply to others. These "multilingual issues" concern the very definition of a word and its structure, and they must be addressed before any parsing algorithm can be applied effectively. The main issues are **tokenization, word segmentation, and morphology.**

1. Tokenization, Case, and Encoding

Tokenisation is the process of breaking a sentence into a sequence of tokens, which is more complex than simply splitting by spaces.

- **Tokenization:** The definition of a "word token" can vary between treebanks and parsers. In English, for example, contractions and possessives are often split into multiple tokens. The Penn Treebank treats *today's* as two tokens: today and 's. This is because the 's marker (as a possessive or a copula verb) can apply to an entire constituent, not just the preceding word.

  Example: Possessive Marker on a Constituent

  (NP (NP (NP First)

(PP of

(NP America)))

's)

operating results)

- **Explanation:** In this example, the possessive marker 's applies to the entire noun phrase "First of America." Separating 's as its own token is necessary to capture this syntactic relationship correctly.

- **Case:** Uppercase and lowercase letters provide valuable information. For instance, an unseen word like *Boeing* is more likely to be a proper noun due to its initial capital letter than a verb like *singing*. A simple strategy of lowercasing all text can lead to information loss. To handle this, parsers can use case information, for example by replacing low-frequency capitalized words like *Patagonia* with a generalized pattern like Xxx to better handle unknown proper nouns.
- **Encoding:** For non-ASCII languages, character encoding must be handled consistently. A parser and its training data (treebank) must use the same encoding (e.g., UTF-8, GB, BIG5 for Chinese). Inconsistencies, even with a simple character like a period (.), can cause errors.

2. Word Segmentation

Many written languages, such as Chinese, do not use spaces to separate words. **Word segmentation** is the process of identifying word boundaries in a continuous stream of characters. This is a critical prerequisite for parsing.

- **Ambiguity in Segmentation:** The same character sequence can often be segmented in multiple ways, leading to different meanings.

  Example: Chinese Segmentation Ambiguity

  Given the text 北京大学生比赛, two possible segmentations are:

  1. 北京 (Beijing) / 大学生 (university students) / 比赛 (competition)

     → *"Competition among university students in Beijing."*

2. 北京大学 (Beijing University) / 生 (gives birth to) / 比赛 (competition)

   → *"Beijing University gives birth to competition."* (less plausible)

- **Impact on Parsing:** If segmentation is performed as a separate first step, any errors will be passed to the parser, which cannot correct them. A better approach is to use a **word lattice** (a finite-state automaton representing all plausible segmentations) as input to the parser. This allows the parser to choose the segmentation that results in the most probable overall parse tree.

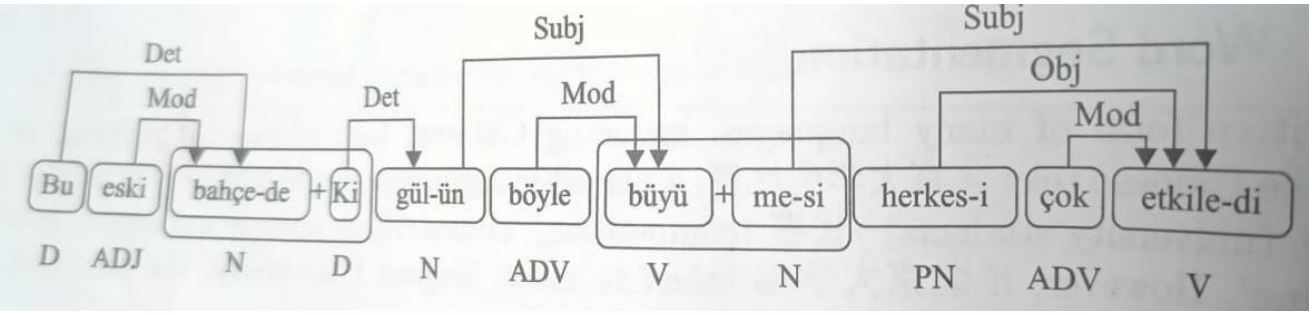
3. Morphology

In many languages, words are not atomic units but are composed of smaller meaningful parts called **morphemes** (a stem and affixes). This has significant implications for parsing.

- **Agglutinative and Inflectional Languages:**
  - In agglutinative languages like **Turkish**, morphemes are chained together to form very complex words. Syntactic dependencies often exist between these morphemes, not just between the words themselves.

Diagram: Turkish Dependency Analysis

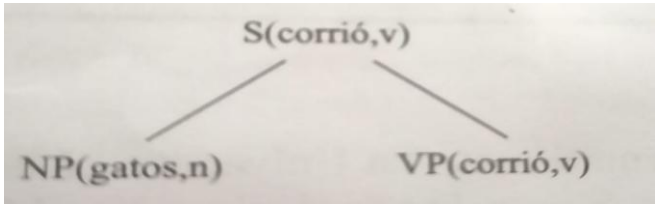
The below diagram of a Turkish dependency parse shows that morphemes (separated by +) are the true heads and dependents. For example, the morpheme ki depends on bahçe-de (garden-in), and the verb morpheme büyü (grow) is modified by the adverb böyle (so). A parser for Turkish must be aware of this internal word structure.



- In inflectional languages like **Czech** or **Spanish**, words change their form (inflect) to mark grammatical properties like case, gender, and number. This leads to a massive number of word forms (e.g., 336 forms for a Czech adjective), causing data sparsity and morphological ambiguity.

- **Adding Morphological Awareness to Parsers:**
  - This can be handled by treating morphology as a complex **POS tagging** task, where the tag encodes rich morphological information (e.g., V--M-3---- for a 3rd person masculine verb).
  - In **discriminative models**, morphological features can be used to improve accuracy. For example, a parser for Spanish could learn that a plural noun *gatos* (cats) is unlikely to modify a singular verb *corrió* (ran), preventing an incorrect dependency.

Diagram: Spanish Parse Fragment



A morphologically-aware parser would penalize the dependency between the plural noun NP(gatos,n) and the singular verb in VP(corrió,v), thus choosing a more grammatically plausible analysis.

In a generative model for **Korean**, it was found that using word **stems** rather than full word forms or suffixes for calculating dependency probabilities helped the parser generalize better over morphologically complex words and improved parsing accuracy. This demonstrates that correctly modeling morphology is key to building robust parsers for such languages.

UNIT – III

Semantic Parsing-I

Q.1 What is semantic parsing?

Semantic parsing is an important phase of Natural Language Processing (NLP). any document is comprised of a set of sentences. These sentences are formed by the words arranged according to the rule-based grammar as per the construe in a particular language. Extracting the meaning out of these syntactically arranged sentences is the objective of semantic parsing. Parsing is analyzing a text into logical syntactic components and semantics is study of meaning. The information pieces are identified and related in semantic parsing. The meaningful parts are identified in the text, and they are transformed into suitable data structures for high level task accomplishment.

Q.2 What are types of semantic parsing?

or

Differentiate between deep semantic parsing and shallow semantic parsing.

Semantic parsing is about identifying the meaning. There are two approaches which can be adapted by extracting the meaning out of text.

1. Deep semantic parsing.
2. Shallow semantic parsing.

Deep semantic parsing:

Consider domain specific applications like air travel reservation, football gaming simulation, querying geographic database etc.

In these kinds of applications precise and each meaning extraction can be done based on the restricted limits of the domain.

In this approach based on the query or the purpose, output is generated from the meaning representation. approach is known as deep semantic parsing.

In the case of deep semantic parsing there is little or no scope of reusability as every domain is unique and requires different understanding and representation of the concepts.

Shallow semantic parsing:

The second approach is not domain specific and is more generic. In this approach the task of meaning representation is divided into small pieces. These pieces are responsible for capturing small manageable components that represent meaning. It is known as shallow semantic parsing.

The related sets of meaning representations are created in this case, for example extraction of word sense disambiguation followed by predicate argument structure.

In the case of shallow semantic parsing as generic meaningful pieces need to be created, it is very difficult to have general purpose ontology and symbols which are shallow from learning point of view but have high reusability irrespective of the applications.

Q.3 What is semantic Interpretation?

Semantic interpretation facilitates the joining of different components which define meaning representation of the text. This representation, when fed to a computer, can be further processed for computational manipulations and

search for any application. Semantic parsing is a part of semantic interpretation.

Q.4 Explain semantic theory.

The semantic theory was proposed by **katz** and **fodor** in 1963. It addresses the fact of meaning representation which is lacking in Chomsky's transformational grammar.

Following points are stated in semantic theory:

1. Ambiguous sentences should be taken care of and explained properly.
   For example: Consider the word 'bank' in the sentence 'I went to bank'. In this sentence ambiguity exists as there can be two meanings of the word bank first is money bank and second is riverbank.
2. Ambiguity resolution of the words depending on context should be done.
   For example: If the sentence is extended as 'I went to bank to withdraw money' then the theory should be able to disambiguate word 'bank' to extract correct meaning.
3. The theory should be able to identify meaningless but syntactically correct sentences.

   For example: Consider the sentence 'Colorless green ideas sleep furiously'. This sentence does not carry any meaning but is syntactical correct.

   Many times, the same semantic content can be represented by different unrelated syntactic structures such sentences should be handed.

Q5. List and explain all the components in semantic interpretation.

The process of semantic interpretation consists of different components to represent a text which can be fed into and processed by computer for undergoing various functionalities like search which are basis of language understanding system.

Following are the major components in this process

1. Structural ambiguity:

- Any sentence is represented by its syntactic structure.
- Syntax and semantics are closely related to each other, and we consider that the semantic interpretation is based on underlying syntactic structure.
- So syntactic processing is the first step in semantic interpretation.

2. Word sense:

- Many times, in any language the same word is used in different meaning depending on the world knowledge.
- For example, the word "bank" represents a money bank, or it can also possess a meaning riverbank.
- Due to inherent intelligence and language vocabulary by humans it is not difficult for them to understand the meaning of the word expected by speaker or author.

  Consider the examples:

  1. Go to the bank to withdraw money.

  2. Fetch some water from riverbank.

- It's an easy task for humans to disambiguate the meaning of word bank in above sentences.

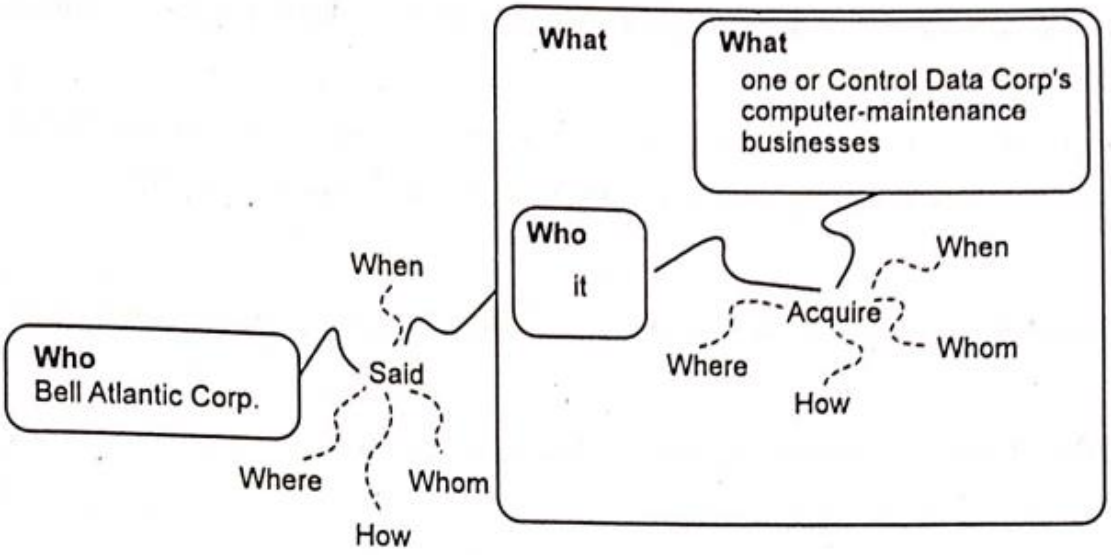
- But for a machine this word sense disambiguation is a challenging task, and it plays an important role in semantic interpretation.

3. Entity and entity resolution:

- Any text consists of various entities falling in different categories like person name, locations, quantities, etc.
- Identification of these entities in the major task in semantic interpretation system.
- Named Entity Recognition (NER) is a subtask of information extraction for locating and classifying named entities in the above-mentioned categories.
- **For example:** [Devika] person bought 100 shares of [Infosys] organization in [2019] time
- Another important task is co-reference resolution. co-reference occurs when two or more expressions in text refer to the same person.

  For example: Pranjali said she will sing. In this sentence proper noun is Pranjali and she refers to same person. This task is also under information extraction and a major component in semantic interpretation.

4. Predicate argument structure:



Q.5.1 : A representation of *who* did *what* to *whom*, *when*, *where*, *why* and *how*

- After finishing all the above-mentioned tasks of word sense disambiguation, NER and coreference resolution, next task is to identify the participants in the events.
- It is important to identify what is the role of entity in a particular event.
- This is known as resolving the argument structure of predicate in a sentence.
- It is typically identification of who did what to whom, when, where and how as shown in the above figure.

5. Meaning representation:

- This is the last step in semantic interpretation and called deep representation.
- It involves building meaning representation which can be used by algorithms for various applications.

- Research is still going on in general-purpose representations.
- Much study has been done in domain specific applications.

For example: Consider a Geoquery domain, the example query can be represented as which river is the longest?

Answer (x_1, longest (x, river (x_1)))

System Paradigms

Q.6 What are the different approaches for practical implementation of semantic interpretation?

- Based on the diversity of languages and levels of granularity and generality there can be different approaches to handle semantic representation.
- The main constraint is posed by lack of data availability of the languages.
- Due to these constraints some successful approaches for practical implementation of semantic interpretation fall in three categories:

1. System architecture:

1. Knowledge based: These systems are designed using predefined set of rules for finding the solution to a particular problem.

2. Unsupervised: Solution to a particular application is found by minimum human involvement.

3. Supervised: This technique involves model training by application of different machine learning algorithms. Feature functions are created to create features which are used to predict labels to handle unseen data.

4. Semi supervised: Supervised technique involves more human involvement so it is expensive. So, the data set can be expanded by application of machine generated output directly.

2. Scope

1. Domain dependent: Systems designed for specific domains such as travel reservations or football coaching.

2. Domain independent: It includes generic techniques applicable for multiple domains.

3. Coverage:

1. Shallow: In this approach intermediate representation is generated to be used by machines.

2. Deep: Through this approach representation, which is created, directly used by machine.

Q.7 Write a note on availability of resources for word sense disambiguation.

OR

What are the different resources for word sense disambiguation in language understanding?

- In the case of natural language understanding the availability of the corpus is the crucial factor.
- Up till now the inadequacy of the tagged sense data is the major issue faced by language understanding systems.
- In the case of word sense disambiguation, the resources have evolved from machine readable dictionaries to the use of word net in the recent systems.

- In early days Longman Dictionary of Contemporary English (LDOCE) and Roget's Thesaurus were the resources used for task of disambiguation.
- In the late 1980's WordNet was developed, which contributed significantly in this task.
- WordNet is a powerful resource which contains a lexical database of word senses with multiple parts of speech of a language.
- It also handles different relationships like hypernymy, homonymy, metonymy etc. connecting different words.

Word Sense Systems

Q.8 Explain how word sense disambiguation can be done by rule-based systems using Lesk algorithm.

- In early days word sense disambiguation used to done using dictionary sense definitions.
- These are the man-made resources and now this has become historical data which is available in archived publication.
- Even though this information is not available today, some algorithms and techniques are still useful. The first and very simple algorithm was proposed by lesk and these first-generation algorithms are based on computerized dictionaries.
- Lesk algorithm is further simplified as shown in Fig. Q.8.1 Algorithm 3.1.

Algorithm 3.1: Pseudocode of the simplified Lesk algorithm

The function COMPUTEOVERLAP returns the number of words common to the two sets

Procedure: SIMPLIFIED_LESK (word, sentence) returns best sense of word

1. best-sense ← most frequent sense of word
2. max-overlap ← 0
3. context ← set of words in sentence
4. **for all** sense Є senses of word do
   signature← set of words in gloss and examples of sense
   overlap← COMPUTEOVERLAP (signature, context)
5. **if** overlap gt max-overlap **then**
   max-overlap ← overlap
   best-sense ← sense
6. end if
7. end for
8. return best-sense

Fig. Q.8.1 Algorithm 3.1

Q.9 Explain the working of dictionary-based algorithm by Yarowsky using Roget's Thesaurus categories.

OR

Explain the algorithm for disambiguating words in Roget's Thesaurus categories.

This dictionary-based algorithm uses Grolier's Encyclopedia. It classifies unseen words in 1042 categories based on statistical analysis of 100-word lists in alphabetical order.

The algorithm is as shown in Fig. Q.9.1.

1. Collect contexts for each of the *Roget's Thesaurus* categories.
2. Determine weights for each of the salient words in the context.

$$\frac{P(w_i \mid RCat)}{P(w_i)}$$

3. Use the weights for predicting the appropriate category of the word in the test corpus.

$$\underset{RCat}{\arg\max} \sum_{w} \log \frac{P(w_i \mid RCat)\, P(RCat)}{P(w_i)}$$

Fig. Q.9.1 : Algorithm for disambiguating words into *Roget's Thesaurus* categories

This algorithm works as follows:

- In step 1 contexts are collected.
- In the next step the weight for each word is computed. For these 50 words are used for each side of context word.
- P(w / RCat)=Probability of occurrence of word w in context of Roget's Thesaurus category RCat.
- In the final step unseen words in test set are classified into categories which maximum weight.

Q.10 Explain Structural Semantic Interconnections (SSI) algorithm for word sense disambiguation.

The algorithm is designed by Navigli and Velardi and uses graphical representation for word sense disambiguation.

Consider the example shown in Fig. Q.10.1 for the noun bus with a vehicle sense and connector sense.

The algorithm fetches the information from following resources to understand the context of given word

o WordNet

o Domain labels

o Annotated corpora to generate semantic graph.

The SSI algorithm consists of two steps:

1. An initialization step

2. An interactive step.

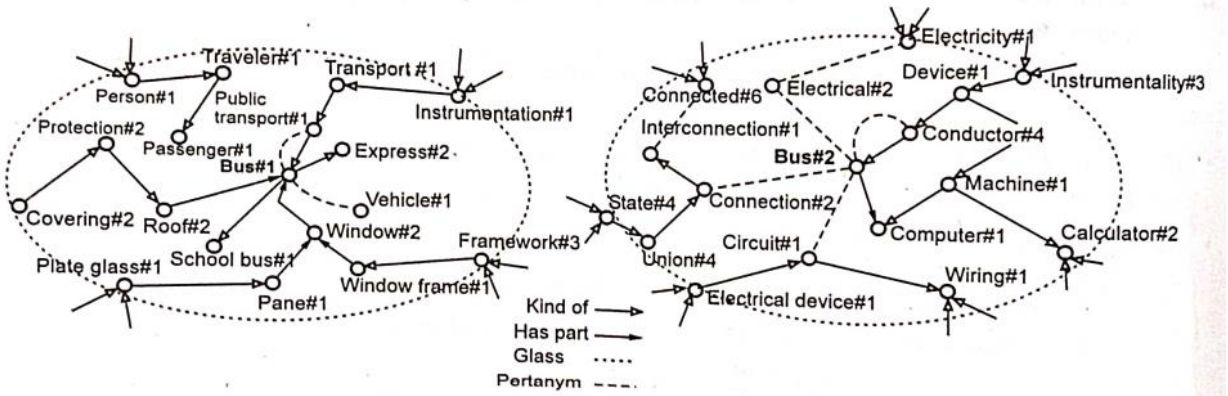


Fig. Q.10.1 : The graphs for sense 1 and 2 of the noun *bus* as generated by the SSI algorithm

The algorithm disambiguates the word of a particular context, till further disambiguation is not possible.

The algorithm has notations as shown in Fig.Q.10.2

Notation :

- T Notation (the *lexical context*) is the list of terms in the context of the term t to be disambiguated. $T = [t_1, t_2, ..., t_n]$.
- $S_1^t, S_2^t, ..., S_n^t$ are structural specifications of the possible concepts (or, senses) of t.
- *I* (the ***semantic context***) is the list of structural specifications of the concepts associated with each of the terms in $T \setminus \{t\}$ *(except t)*. $I = [S^{t_1}, S^{t_2}, ..., S^{t_n}]$, that is, the ***semantic interpretation of T***.

- *G* is the grammar defining the various relations between the structural specifications (or ***semantic interconnections***) among the graphs.
- Determine how well the structural specifications in *I* match that $S_1^t, S_2^t, \ldots, S_n^t$ using *G*.
- Select the best matching S_i^t.

Fig. Q.10.2

- It works as follows
  - Great set of pending terms in context
    $P = \{t : | S^{ti} = \text{null}\}$
  - Use I to disambiguate P in each iteration.
  - As the algorithm proceeds, which each iteration, one term in P is disambiguated and removed from pending list.
  - It stops when no more terms remain for disambiguation.
  - Output I is updated with sense of t.
- Select terms t in P having semantic interconnections which one sense of S of t and one or more senses in I.
- Function f I (S, t) is computed to determine likelihood of S to be correct interpretation of t

$$f_I(S, t) = \begin{cases} P\left(\{\Psi(S,S') \mid S' \in I\}\right) & \text{if } S \in \text{Senses (t)} \\ 0 & \text{Otherwise} \end{cases}$$

Where,

Senses (t) = Senses associated with t

$$\Psi(S, S') = P'(\{\omega(e_1 \cdot e_2 \ldots e_n) \mid S \xrightarrow{e_1} S_1 \xrightarrow{e_2} \ldots \xrightarrow{e_{n-1}} S_{n-1} \xrightarrow{e_n} S'\})$$

Function P′ of weights ω connecting S and S′ where S and S′ are semantic graphs are e to e_n are edges.

- At last CFG

G = (E, N, SG, PG) includes meaningful semantic patterns

where $E = \{e_{\text{kina - of}}, e_{\text{has - kind}}, e_{\text{part - of}}, \ldots\}$ are the edge labels.

$N = \{S_{or}, S_s, S_g, S_1, S_2, \ldots E_1, E_2, \ldots\}$ are the nonterminal symbols.

S_G = Start symbol of graph.

$P_G = \{S_G \rightarrow S_S \mid S_g, S_S \rightarrow S_1 \mid S_2 \mid S_3, S_1 \rightarrow E, S, \mid E_1,$

$E_1 \rightarrow e_{\text{kind - of}} \mid e_{\text{part - 01}}, S_g \rightarrow e_{\text{glass}} S_5 \mid S_4 \mid S_5, \ldots\}$

Q.11 Explain how Word Sense Disambiguation (WSD) can be done by supervised approach.

- In supervised approach the complexity of WSD system is handled by machine learning techniques but uses data annotation as well which is the process of adding metadata in the form of tags to the data.
- The words present in a particular corpus are first manually disambiguated.Various features are extracted for these words.
- Then the supervised system is designed which has machine learning classified or trained on these features.
- The advantage of this system is users can form and update rules in the form of features and generate training data.
- Many researchers have explored various techniques to address WSD problems starting from machine learning extended to the use of decision lists which uses rich set of features in machine learning framework.
- The researchers further enhanced these features to incorporate different context levels i.e. sentences, paragraphs, micro context etc.

Q.12 What are the different classifiers and features used in supervised WSD?

The popular and efficient classifiers include

1. Support Vector Machines (SVMs):

Support Vector Machine (SVM) model follows supported machine learning approach. It performs well with a limited amount of data.

2. Maximum Entropy (MaxEnt) Classifier:

It is a probabilistic classifier which belongs to an exponential model.

Features of supervised approach:

The more commonly found subset of features that have been useful in supervised learning of word sense.

Lexical context: this feature comprises the words and lemmas of words occurring in the entire paragraph or a smaller window of usually five words.

Parts of speech: this feature comprises the POS information for words in the window surrounding the word that is being sense tagged.

Bag of words: this feature comprises using an unordered set of words in the context window.

Local collocations: local collocations are an ordered sequence of phrases near the target word that provide semantic context for disambiguation.

Topic feature: the broad topic, or domain of the article that the word belongs to is also a good indicator of what sense of the word might be most frequent.

Algorithm: rules for selecting syntactic relations as features.

If w is a noun then

select parent head word (h)

select part of speech of h

select voice of h

select position of h (left,right)

else if w is a verb then

Select nearest word l to the left of w such that w is the parent head word of l

Select nearest word r to the right of w such that w is the parent head word of r

select part of speech of l

select part of speech of r

select part of speech of w

select voice of w

else if w is adjective then

select parent head word (h)

select part of speech of h

end

Q.13 Explain how unsupervised approach can address various issues posed by supervised approach in the problem of word sense disambiguation.

In case of supervised approach, the Word Sense Disambiguation (WSD), process is obstructed by lack of labelled training data for training a classifier to handle every sense of each word in a particular language.

This issue can be resolved by following solutions:

1. There can be several instances or occurrences of a word. Based on these instances the clusters are formed covering examples of words of a certain sense. This is called sense induction through clustering.

2. Create a matrix for identification of nearness of a given instance of the word, with sets of known senses of a word. Based on this select the closest sense to that instance.

3. Consider some examples of words of certain sense. Use these seeds, perform iterations on these seeds and enhance the clusters.

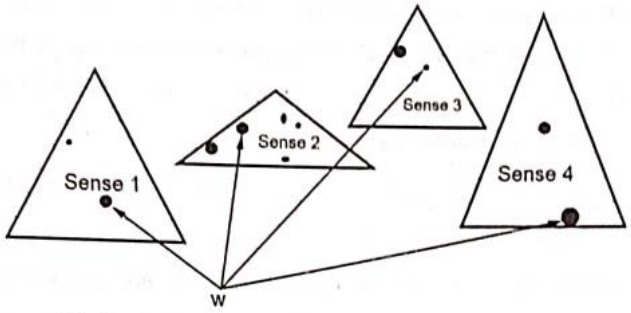
Q.14 What are different algorithms which use distance measures to identify senses?

OR

Explain different algorithms of word sense disambiguation by unsupervised approach.

There are many algorithms designed by various researchers which use distance measures to identify cases. Some of the algorithms are explained below:

1. Rada proposed an algorithm which uses a metric to compute shortest distance between two pairs of senses in WordNet.
   For example, IS-A in WordNet which assumes that different co-occurring words may possess the senses which minimize the distance in semantic network.
2. Resnick proposed the measure of semantic similarity known as information content in IS-A taxonomy.
3. Agirre and Rigau proposed the approach which extended information content measure to a measure called conceptual density as shown in Fig. Q.14.1.



Word to be disambiguated : W
Context words : w1 w2 w3 w4

Fig. Q.14.1 : Conceptual density

As shown in Fig. Q.14.1 sense 2 has highest conceptual density so it is chosen as a correct sense.

Conceptual density can be computed as,

$$(D, C_c, m) = \frac{\sum_{i=0}^{M-1} \text{Hyponyms}^{i-0.20}}{\text{Descendants}_c}$$

4. Resnik proposed a measure for computation of sense of a word based on predicate-argument stastistics.

A_R is considered to be a selectional association of predicate P to concept C with argument R. A_R is defined as,

$$A_R (P, C) = \frac{1}{S_R (P)} P (C|P) \log \frac{P (C \mid P)}{P (C)}$$

For noun n, arguments relation R and $\{S_1, S_2, \ldots S_R\}$ as possible senses the

$$C_i = \{C \mid C \text{ is an ancestor of } S_i\}$$

$$a_i = \max_{c \in C_i} A_R (P, C)$$

for $i = 1$ to K where a_i is score of sense S_i

5. Leacock, Miller and Chodorow proposed an algorithm which used corpus statistics and WordNet relations.

Q.15 Explain the algorithm based on crosslinguistic information.

OR

Explain SALAAM algorithm.

This algorithm is an unsupervised algorithm based on crosslinguistic information.

Along with word sense disambiguation it is also helpful in language translation system, as it also figures out line difference in senses that required translating to other languages.

The algorithm is explained in Fig. Q.15.1

1. L1 words that translate into the same L2 word are grouped into clusters.
2. SALAAM (Sense Assignment Leveraging Alignment and Multilinguality) identifies the appropriate senses for the words in those clusters according to the words senses' proximity in WordNet. The word sense proximity is measured in information theoretic terms on the basis of an algirthm by Resnik .
3. A sense selection criterion is applied to choose the appropriate sense label or set of sense labels for each word in the cluster.
4. The chosen sense tags for the words in the cluster are propagated back to their respective contexts in the parallel text. Simultaneously, SALAAM projects the propagated sense tags for L1 words onto their L2 corresponding translations.

Fig. Q.15.1 : SALAAM algorithm for creating training using parallel English-to-Arabic machine translations

Q.16 Explain how WSD can be done using a semi supervised approach by Yurovsky algorithm.

OR

Explain now Yurovsky algorithm addresses WSD problem.

- In the case of semi supervised approach the process of WSD starts from a small seed of examples and an iterative algorithm that identifies more training examples using a classifier.
- This additional, automatically labeled data can then be used to enhance the training data of the classifier to provide better predictions for the next selection cycle, and so on.
- One such semi-supervised algorithm is the Yurowsky algorithm.

- The algorithm is based on the assumptions that two strong properties are exhibited by corpora.

1. One sense per collocation:

syntactic relationship and the types of words occurring nearby a given a word tend to provide a strong indication as to the sense of that word.

2. One sense per discourse:

Usually, in each discourse, all instances of the same lemma tend to invoke the same sense.

The algorithm starts with a large untagged corpus. It then identifies the examples of polysemous words (words or phrases having multiple meanings) and stores are relevant sentences as lines.

To understand the working let's consider the example.

Consider a target word plant based on manual choice of possible senses a seed set can be formed for example in our case seed set for word plant can be {life, manufacturing}.

The first word relates to flora and the second word to industry is classified according to rules.

Consider U as a set of unlabeled examples, as shown in Fig. Q.16.1.

In the next step of instances of two senses / and m according to the seed set are added to the set as shown in figure.

This will be done iteratively till no new examples are found.

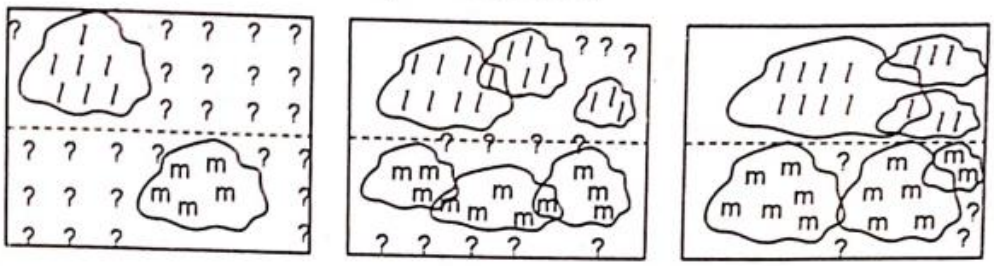


Fig. Q.16.1 : Yarwsky's algorithm : with every iteration new examples are labelled according to seed set {life (*l*)}, manufacturing (m)}

The algorithm can be written as follows.

Step 1: In a sufficiently large corpus, identify all the instances of a particular polysemous word that needs to be disambiguated, storing its context alongside.

Step 2: Identify a small set of instances that are strongly representative of one of the senses of the word. This can either be done in a completely unsupervised fashion by identifying collocations that give a strong indication of the sense usage for the word under consideration or by manually tagging a small portion of the data. In this example, we assume a polysemous word with only two senses, but this algorithm can be extended to n senses.

Step 3:

Step 3a: Train a supervised classifier on this set of examples.

Step 3b: Using these classifiers, classify the remaining instances of the word in the corpus and select those that are classified above a certain level of confidence.

Step 3c: Filter out the possible misclassifications using one sense per discourse constraint and identify possible new collocations to be added to the list of seed collocations.

Step 3d: Repeat step 3 iteratively, thereby slowly shrinking the residual.

Step 4: Stop. At some point, a small, stable residual will remain.

Step 5: The trained classifier can now be used to classify new data, and that in turn can be used to annotate the original corpus with sense tags and probabilities.

Q.17 Explain the Mihalcea and Moldovan algorithm.

OR

explain the algorithm for generating examples for words tagged with senses.

The system based on Mihalcea and Moldovan algorithm addresses the problem of lack of hand cannot data to train supervised system.

It works as an example of a particular sense from a large corpus in WordNet.

The algorithm works as follows:

1. Extract: In wikipedia the sentences may contain words as simple links.

For example: [bar]] or a piped link. Piped link is an internal link with wikitext that creates hyperlink (underline, clickable) text displayed on a wikipedia page that is different from the title of the page to which the text links.

For example: [Train station | Station]] displays as station but link to the train station Wikipedia article. In these steps all such sentences are extracted for a given word.

2. Filter: If the link is pointing to disambiguation page, further information is needed to disambiguate the word so filter all such links. If link to

disambiguation page is not present, then consider the word as label for piped links string before pipe is a label.

3. Collect: Map all these labels to possible WordNet senses. If they map to various categories of Words they provide sense disambiguated data for training.

The algorithm is described as follows.

Step 1: Preprocessing

For each sense of the word W, determine the synsets of WordNet in which it appears. For each such synset, determine monosemous words included in that synset. Parse the gloss definition attached to each synset.

Step 2: Search

From search phrases using the following procedures in order of preference

1. If they exist, extract monosemous synonyms from the synsets selected in step 1.

2. Select each of the unambiguous parsed constituents in the gloss as a search phrase.

3. After parsing the gloss, replace all stop-words with a NEAR operator and create a query from the words in the current synset. For example, if the synset for produce #6 is grow, raise, farm, produce, and the gloss is cultivated by growing, then the query will look like: cultivate NEAR growing AND (grow OR raise OR farm OR produce).

4. Use only the head phrase combined by words in the synset using the AND operator. For example, if the definition for company #5 is band of people and its synset is (party, company), then the query becomes: band of people AND (party OR company).

Search the Internet with the phrases determined in the previous step and gather matching documents.

From these documents, extract the sentences containing these words.

Step 3: Postprocessing

Keep only those sentences in which the word under consideration belongs to the same part of speech as the selected sense and delete the others.

Q.18 What are the different software programs available for word sense disambiguation.

Several software programs are made available by the research community for word sense disambiguation, ranging from similarity measure modules to full disambiguation systems. Following are some software programs explored for word sense disambiguation.

1. IMS (It Makes Sense):

- This is a complete word sense disambiguation system.
- It is a supervised English all words WSD system.
- This software uses linear support vector machines and use of various knowledge-based features.
- This system facilitates users to integrate different preprocessing tools, classifiers and additional features.
- It can be accessed through http://nlp.comp.nus.edu.sg/software.

2. WordNet Similarity -2.05:

- The WordNet similarity modules for perl provide a quick way of computing various word similarity measures.

- It incorporates the measures of Resnik, Lin, Jiang-conrath, Leacock Chodorow, Banerjee - Pederson and Patwardhan - Pedersen.
- The input to the modules is pair of words and the output is a numeric value indicating their degree of similarity or relatedness.
- It can be accessed through

  http://search.cpan.org/dist/WordNet-Similarity

3. WikiRelate:

- WikiRelate is a system developed by Michael struck and simone Paolo Ponzetto.
- This is a word similarity measure based on the categories in Wikipedia.
- It is used for computing relatedness of the words in a structured way and has more coverage than WordNet.
- The dataset used is wikipedia for computation of semantic relatedness and it is compared to WordNet on various benchmarking datasets.
- It can be accessed through:

  http://www.h-its.org/english/research/nlp/download/wikipedia-similrity.php